Component hierarchy (render order)

> **Live demo:** `/component-hierarchy` · **Code:** `src/app/component-hierarchy/`

What it is

When a route segment has several special files, Next.js nests them in a**fixed order**. Knowing this order explains *which* boundary catches an error, *what* shows during loading, and *where* your page actually renders.

The order (outermost → innermost)

```
layout.js
└── template.js
    └── error.js       (React error boundary)
        └── loading.js   (React Suspense boundary)
            └── not-found.js (boundary for notFound())
                └── page.js  (or a nested layout.js, recursively)
```

So a `page` is wrapped by `not-found`, which is wrapped by `loading`, wrapped by `error`, wrapped by `template`, wrapped by `layout`.

Why the order matters

- **`error` is *outside* `loading` and `page`** → it can catch errors thrown while the page (or its loading state) renders.
- **`loading` is *outside* `page`** → the Suspense fallback shows while the async page resolves.
- **`layout` / `template` are outermost** → they stay on screen while inner parts swap. (Layout persists; template re-mounts — see `/routing-files`.)

Nested routes

For nested segments this repeats**recursively**: a child segment's entire stack renders inside the parent segment's `page` slot (where the child's `layout` sits). Parent layouts wrap child layouts wrap pages.

Interview Q&A

- **Q: In what order are the special files nested?**layout → template → error → loading → not-found → page.
- **Q: Why can `error.tsx` catch a `page.tsx` error but not a sibling `layout.tsx` error?** Because `error` wraps `page` (it's outside it) but is *inside* its own segment's `layout` — so it can't wrap that layout. The root layout needs `global-error.tsx`.